Layers as Lenses: A Narrative of Feature Learning in Deep Networks

Anonymous Authors[1]

Abstract

Deep neural networks with increasing depth consistently outperform their shallower counterparts across a wide range of architectures and datasets. However, a principled theoretical explanation for this phenomenon remains at a superficial level. While explanations such as representational power, input space folding, optimization conditioning, the neural tangent kernel and hierarchical feature learning have been proposed, they all have glaring deficiencies. We develop a new narrative that postulates the following. From the viewpoint of any given layer, the other layers act as a lens that applies importance weights to the input, facilitating the emergence of features without global scope that would otherwise be overlooked. Gradient methods simply update the parameters of a layer to best fit the data that is importance-weighted by a lens made of other layers. Individual neuron parameters serve a dual role of 'separator for data lensed by other layers' and 'component of lens for other layers'. This enables a positive feedback loop that forms the core mechanism of feature learning under gradient descent in deep networks. We support our narrative with theoretical analysis and empirical results on the recently proposed Deep Linearly Gated Network (DLGN), an architecture combining elements of deep linear and ReLU networks. We also show how our analysis leads to a natural notion of hierarchical feature discovery in the feature learning regime.

1. Introduction

Despite the empirical successes of deep neural networks, prevailing explanations for their learnability remain largely superficial. There exists several popular narratives – such as "deeper networks represent more complex functions," "finite-width deep networks approximate infinite-width kernel methods," or "feature kernels evolve to become better during training" (see, e.g., (Nichani et al., 2023; Arora et al., 2019; Frei et al., 2023)). These accounts overlook why such structures can be reliably learned, fail to address where these explanations break down, and do not explain why empirical phenomena routinely deviate from theoretical predictions. They offer at best, limited insight into some key questions – (1) Why does gradient-based optimization lead to effective solutions? (2) Why and how do multiple layers enable this behavior? (3) How is the curse of dimensionality avoided?

A more expressive theory of feature learning is needed to move beyond the current surface-level intuitions and reveal the mechanisms that truly underpin deep learning.

In this paper, we restrict our focus to binary classification problems with feature vector data. Extensions to more complex tasks – such as generative modelling or structured inputs like images and text – are beyond the scope of this work. We will also restrict our theoretical arguments to the infinite training data and infinitesimal learning rate setting (corresponding to population gradient flow rather than stochastic gradient descent) for convenience. We strongly believe our arguments can be extended to the SGD setting, but such an extension is beyond the scope of this work. The experiments to support our theoretical arguments are naturally done with finite data and finite learning rate.

2. Literature Review

2.1. An Overview of Current Narratives

Neural Networks and the the Kernel regime: An important tool for studying the optimization and generalization capabilities of neural networks has been the Neural Tangent Kernel (Jacot et al., 2018; Lee et al., 2019). In the infinite-width limit and a certain scaling the network evolution is equivalent to kernel regression given by the NTK. Under this regime, the NTK remains unchanged, making it easier to study the learning dynamics of neural networks (Du et al., 2019b;a; Arora et al., 2018). The mean-field parameterization (Mei et al., 2018; Chizat & Bach, 2018; Bordelon & Pehlevan, 2025) has been a popular alternative to the NTK parameterization since the features evolve under this regime.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Feature learning and gradient descent: A growing body of work attributes the empirical gap between neural networks trained by gradient descent and kernel methods to the networks' ability to learn task-relevant features that fixed kernels cannot capture (Allen-Zhu & Li, 2019; Daniely & Malach, 2020; Li et al., 2020; Yehudai & Shamir, 2019). Even a small number of gradient steps on the first layer of a two-layer network produces features that improve downstream performance (Daniely & Malach, 2020; Damian et al., 2022) . These results are particularly relevant for data that is high dimensional but have a low-dimensional latent structure: in such settings, feature learning effectively reduces the problem to learning the latent subspace. This results in staircase training dynamics formalized in (Abbe et al., 2021; 2022; 2023). Empirical investigations show that neural networks discover coarse/simple features first and then they progressively capture finer/complex features, consistent with the theoretical staircase picture (Kalimeris et al., 2019). A complementary analytical tool has been to study the evolution of the data-dependent NTK: unlike the fixed NTK of the infinite-width limit, the empirical NTK changes during finite-width training and tends to align with the target, a behavior labeled kernel alignment (Fort et al., 2020; Loo et al., 2022; Atanasov et al., 2022; Wang et al., 2023).

The advantage of depth: Understanding how depth compositionally enhances feature learning remains a central open question. Deeper networks are hypothesized to build a hierarchical representation of features, where the simple features learned by early layers are composed into more abstract concepts by later layers (Allen-Zhu & Li, 2020). (Telgarsky, 2016) show that deep networks cannot be approximated by shallower networks, unless they have an exponentially large number of nodes. (Montufar et al., 2014) show that depth enables models to learn a larger number of linear regions thus approximating the target function better than shallower models. (Nichani et al., 2023) show that three-layer neural networks have provably richer feature learning capability compared to two-layer networks.

2.2. Pitfalls of Current Narratives

The lazy regime of the NTK (Chizat et al., 2019) does not explain why neural networks are able to learn complex features from the data. Studies focusing on the feature learning or rich regime – while providing interesting/useful results – often consider very simple settings and have restrictive assumptions. For example: only one gradient step, two-layer networks, linear/quadratic activations, layer-wise training schedules, specific initialization scalings, and strong assumptions about the data (Cui et al., 2024; Dandi et al., 2024; Ba et al., 2022; Damian et al., 2022; Daniely & Malach, 2020; Abbe et al., 2022). There is very little understanding on how well these results transfer to deep networks trained end-to-end using gradient descent. The crucial deep learning narrative question – given below – remains unanswered to this date. *How do the multiple layers in a deep neural network interact with each other and jointly adapt during training to discover task-relevant features?* A related question of lesser importance is given as follows: *When an architecture and data setting has multiple functionally different global/local minima, how does the initialization relate to the eventual gradient descent solution?* The latter question is narrower and more technical than the former. Our inability to answer the latter question convincingly for a non-trivial setting makes the former question even harder to grasp.

3. Decision Tree Data and Deep Linearly Gated Network Architecture

3.1. Motivation for the Data Setting

A primary goal in selecting a data setting for theoretical study is to ensure genuine non-triviality; the concept class should be complex enough that feature learning via gradient descent on a deep architecture is necessary. While real-world datasets often exhibit this property – resisting solutions by linear or kernel methods – their inherent opacity renders precise analysis of model dynamics intractable. By contrast, canonical synthetic benchmarks such as parity functions or linearly separable data or polynomials (see, e.g., (Daniely & Malach, 2020; Frei et al., 2023; Soudry et al., 2018; Wu et al., 2023; Abbe et al., 2021) are somewhat simplistic and fail to capture the kind of intricate structure that motivates deep learning theory. Data labeled by an oblique decision tree (Banerjee et al., 2025) however, offers synthetic complexity without analytic opacity, furnishing a non-trivial, structured classification task.

An effective data model should permit generalization of its key mechanisms to broader settings, while remaining amenable to precise analysis. Oblique decision trees occupy this middle ground: although real-world datasets are rarely generated by tree processes, many possess rich, multi-scale hierarchical structure that trees can capture or approximate. The essential elements of a decision tree – its internal node hyperplanes – can be isolated and interpreted as discontinuities in the label function, regardless of the tree context. As a result, properties established for tree-structured data are more plausibly extended to realistic data regimes than results derived from artificial settings such as parity functions or polynomials. Because oblique trees combine structural expressiveness with analytic tractability, they provide a robust foundation for arguments that aim to reflect the mechanisms enabling deep network success in real data.

3.2. Motivation for the Architecture Setting

To meaningfully advance theory, we require an architecture that is empirically competitive with standard deep networks, such as multilayer ReLU models, on challenging tasks. Classic models favored in theoretical work – like deep linear networks or single-hidden-layer neural networks – lack the expressive power and empirical relevance of modern deep architectures. In contrast, the deep linearly gated network (DLGN) achieves close to state-of-the-art performance in feature vector classification tasks (Banerjee et al., 2025) and therefore offers a more faithful proxy for understanding deep feature learning in realistic settings.

A suitable architecture should possess a transparent, layer-wise structure whose parameters can be decomposed into interpretable, semi-independent components. While multilayer ReLU networks have a clear notion of depth, the intricate parameter interactions – especially in intermediate hidden layers – often render isolated analysis of individual weights or neurons meaningless. This challenge is typically avoided in theoretical studies by restricting consideration to shallow (e.g., single-hidden-layer) networks, sacrificing insights into deep feature learning. The DLGN architecture, by design, enables meaningful dissection of parameter dynamics at the level of individual neurons, while preserving the hierarchical depth necessary for examining deep learning phenomena.

3.3. The Oblique Decision Tree Data Setting

The instance $\mathbf{x}$ is drawn uniformly from the surface of a sphere of radius one in $\mathbb{R}^d$. The label $y \in \{+1, -1\}$ is given by a complete, orthogonal and balanced oblique decision tree (COB-ODT according to (Banerjee et al., 2025)) of depth D which recursively partitions the space at each internal node using orthogonal hyperplanes. Concretely, consider a depth $D-1$ complete binary tree with nodes indexed 0 to $N = 2^D - 2$. Each node $i \in \{0, 1, \ldots, N\}$ has a left child $2i+1$, and right child $2i+2$. It is also associated with a 'decision vector' $\mathbf{u}_i \in \mathbb{R}^d$. The $N+1$ decision vectors $\mathbf{u}_0, \ldots, \mathbf{u}_N$ are unit norm and mutually orthogonal. Traversal starts at the root; at each node i, the data point $\mathbf{x}$ proceeds left (right resp.) if the sign of the inner product $\mathbf{u}_i \cdot \mathbf{x}$ is negative (positive resp.). The label $y^*(\mathbf{x})$ is equal to the sign of $-\mathbf{u}_i \cdot \mathbf{x}$ at the node i reached after $D-1$ decisions. The label nodes are given numbers $N+1$ to $2N+2$ for convenience, even though they do not have an associated decision. As a convention, we call the nodes at level $D-1$ as leaf nodes, and the nodes at level D as label nodes, e.g. in the depth $D=4$ tree shown in Figure 1, nodes 7 to 14 are leaf nodes, while nodes 15 to 30 are label nodes.

Let Γ denote the set of paths in the ODT starting at the root 0 and terminating at a label node. Each element $\gamma \in \Gamma$ is a $(D+1)$-tuple. Let $y(\gamma)$ denote the class label corresponding to the path γ. Every element $\gamma \in \Gamma$ is assoicated with a sign vector $\tau(\gamma) \in \{+1, -1\}^D$ giving the sequence of left/right decisions taken. For example, in the tree shown in Figure 1 we have $\tau((0, 1, 3, 7, 15)) = (-1, -1, -1, -1)$.

$$y^*(\mathbf{x}) = \sum_{\gamma \in \Gamma} y(\gamma) \prod_{p=1}^{D} \mathbf{1}(\tau_p \mathbf{u}_{\gamma_p} \cdot \mathbf{x} > 0) \tag{1}$$

where τ_p is used as a shorthand for the p-th element of $\tau(\gamma)$. By construction, for any given $\mathbf{x}$, only one of the paths in Γ has a non-zero contribution in the above expression.

3.4. The Deep Linearly Gated Network Architecture Setting

The deep linearly gated network (DLGN) is defined as a sum of products of sigmoidal activation applied to linear functions. Specifically, the model output is

$$\hat{y}(\mathbf{x}) = \sum_{\pi \in \Pi} a_\pi \prod_{\ell=1}^{L} \phi(\mathbf{w}_{\pi,\ell}^\top \mathbf{x}) \tag{2}$$

where Π is a discrete index set, L is the number of layers and $\phi : \mathbb{R} \to [0, 1]$ is a scaled sigmoid, ($\phi(u) = 1/(1 + \exp(-\beta u))$). The weight vectors $\mathbf{w}_{\pi,\ell}$ form a total of $|\Pi| \times L$ vectors in $\mathbb{R}^d$ and constitute the parameters of the model along with the scalar coefficients v_π.

To establish stronger connections with standard deep neural networks, the construction in (Banerjee et al., 2025) considers $\Pi = [M]^L$ corresponding to all possible paths across a network with L hidden layers and M neurons per layer. Since the number of such paths grows rapidly with depth and width, the model further restricts parameterization via sharing: for $\pi = (m_1, \ldots, m_L) \in [M]^L$, the weight $\mathbf{w}_{\pi,\ell}$ depends only on the index m_ℓ in layer ℓ, i.e. $\mathbf{w}_{\pi,\ell} = \mathbf{w}_{m_\ell,\ell}$.

The scalar coefficients a_π are defined as the product of edge weights along the path π in an associated deep linear network, i.e. $a_\pi = \prod_{l=2}^{L} a_{\ell, m_{\ell-1}, m_\ell}$.

We will refer to the vectors $\mathbf{w}_{m,\ell}$ for some $m \in [M]$ and $\ell \in [L]$ as the gating vector/neuron corresponding to the m-th hidden node in the ℓ-th hidden layer. a_π will be called the scale parameter for path π.

Despite its structural simplicity compared to standard ReLU networks, the DLGN has proven to be a surprisingly powerful learning architecture (Banerjee et al., 2025). It maintains a transparent correspondence between parameters and their effect on the output – each $\mathbf{w}_{i,\ell}$ represents a component whose influence on $\hat{y}$ can be traced directly.

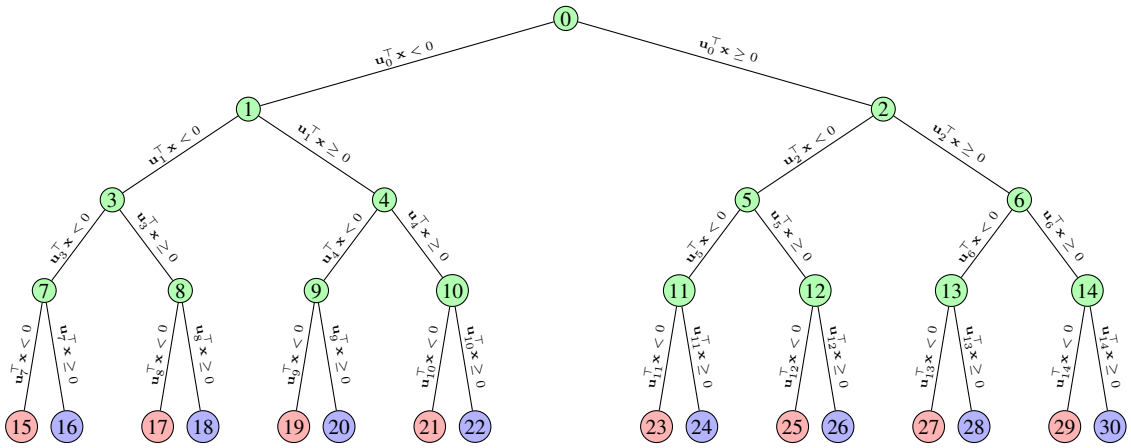

Figure 1. The decision tree with depth $D = 4$ has its internal nodes shaded green. Red leaf nodes are positively labelled and blue are negatively labelled.

4. Learning Dynamics of Single Path DLGN

Consider the single path DLGN, with $M = 1$. While this is not a useful architecture by itself, it still exhibits interesting behavior that is illustrative of the message we intend to convey for deep architectures in general. Precisely, we consider the model

$$\hat{y}(\mathbf{x}; a, W) = a \prod_{\ell=1}^{L} \phi(\mathbf{w}_\ell^\top \mathbf{x})$$

with parameters $a \in \mathbb{R}, \mathbf{w}_1, \ldots, \mathbf{w}_L \in \mathbb{R}^d$. We concatenate the L gating vectors $\mathbf{w}_1, \ldots, \mathbf{w}_L$ together as a single vector $W \in \mathbb{R}^{Ld}$ for convenience. Let $P_\ell : \mathbb{R}^d \to \mathbb{R}^{Ld}$ be such that $P_\ell(\mathbf{u})$ sets the ℓ-th d-dimensional block of the output equal to $\mathbf{u}$ and zero elsewhere.

This model is not powerful enough to fit the ODT labeled data. But the form of the model suggests that it might 'capture' one of the paths of the ODT labeling function. The analysis for establishing this is worthy of a deep dive.

4.1. Lens and Blinders

The main tool that enables our analysis, is the loss gradient with respect to one of the weight vectors (say) $\mathbf{w}_k$. Consider the loss gradient for a single data point $(\mathbf{x}, y^*)$.

$$\begin{aligned} \mathcal{L}(\hat{y}(\mathbf{x})) &= \log(1 + \exp(-y^* \hat{y}(\mathbf{x}))) \\ -\nabla_{\mathbf{w}_k} \mathcal{L} &= \mathbf{E}[\underbrace{\sigma(-y^* \hat{y}(\mathbf{x}))}_{\text{Residue}} \underbrace{\Lambda_k(\mathbf{x})}_{\text{Lens}} \underbrace{\phi'(\mathbf{w}_k^\top \mathbf{x})}_{\text{Blinder}} y^* a \mathbf{x}] \end{aligned} \quad (3)$$

where $\Lambda_k(\mathbf{x}) = \prod_{\kappa \neq k} \phi(\mathbf{w}_\kappa^\top \mathbf{x})$. The negative loss gradient w.r.t. $\mathbf{w}_k$ is a positive scalar multiple of $y^* a \mathbf{x}$.

The gradient descent step for $\mathbf{w}_k$ can now be viewed as a Perceptron-like update over the entire training dataset S, with importance weights to a sample $\mathbf{x}, y$ given by the product of the residue, lens and blinder terms. While $\mathbf{w}_k$ is updated to fit the data that is importance weighted based on its lens, it also acts as a component of the lens for other neurons. With the right lens, the gradient will have a strong component along feature directions that do not have a global scope (like the ODT root node $\mathbf{u}_0$). This dual role of 'component of lens for other layers' and 'linear separator for data lensed by other layers' served by the vectors $\mathbf{w}_k$ forms a positive feedback loop.

4.2. Second Order Analysis

A natural choice for analyzing 'feature learning' is to study how the NTK features change during training, which corresponds to performing a second order analysis. We specialize the analysis of the gradient to the ODT labelled data, where y^* is given by Equation 1 and show how the positive feedback between the two roles of a neuron $\mathbf{w}_k$ enables better learning of the ODT features by the model.

Theorem 4.1. *Let $\mathbf{h}, \mathbf{z}$ be vectors in $\mathbb{R}^d$. Let $a > 0, k \neq \ell$. Let $\mathbf{w}_\lambda^\top \mathbf{h} = \mathbf{w}_\lambda^\top \mathbf{z} = 0$ for all λ. The change in gradient of $\mathcal{L}$ w.r.t. $\mathbf{w}_k$ along $\mathbf{z}$, as $\mathbf{w}_\ell$, moves along $\mathbf{h}$ satisfies*

$$\lim_{\epsilon \to 0} \frac{1}{\epsilon} \Big[-\nabla_{\mathbf{w}_k} \mathcal{L}(W + \epsilon P_\ell(\mathbf{h})) + \nabla_{\mathbf{w}_k} \mathcal{L}(W) \Big] \cdot \mathbf{z} > 0$$

where $\mathbf{z} = s\mathbf{u}_i, \mathbf{h} = \mathbf{u}_j$, $j \in \{N/2, \ldots, N\}$ is a leaf node of the ODT, and i is any ancestor of j. The scalar $s = +1$ if j is in the left sub-tree of i and -1 otherwise. The above inequality also holds when $\mathbf{h}$ and $\mathbf{z}$ are swapped.

An example implication of Theorem 4.1 follows. If $\mathbf{w}_\ell$ moves along $-\mathbf{u}_j$ for some leaf node j on the left side of the root node 0, then $\mathbf{w}_k$ is pushed along $-\mathbf{u}_0$. This tendency of gradient descent to push a neuron based on the movement of another neuron is fundamental to the feature learning dynamics of the DLGN. This enables the model to overcome the curse of dimensionality – i.e. even in a worst case initialization where all the neurons $\mathbf{w}_k$ are orthogonal to all the ODT directions $\mathbf{u}_i$, the neurons evolving under gradient descent have a chance of capturing the ODT features $\mathbf{u}_i$ corresponding to non-leaf nodes i. The gradient at initialization has little to no component along these $\mathbf{u}_i$ at initialization. Without the kind of second order mechanism described by Theorem 4.1 these decision vectors which are crucial for good generalization performance will not be captured. This is much less of a serious problem at lower values of d – the initialization is quite likely to have some component along $\mathbf{u}_i$.

Under a symmetric initialization (like $W = 0, a > 0$), gradient descent will hit a saddle point W^*, corresponding to each of the L neurons taking the same vector $\mathbf{w}^* = c\sum_j -\mathbf{u}_j$, where j ranges over leaf node indices $N/2$ to N. However, the second order dynamics in Theorem 4.1 ensure that with a little asymmetry a positive feedback loop kicks in. A detailed example is discussed in Section 4.3

Second order dynamics of gradient descent in this problem also has a somewhat undesirable behavior.

Theorem 4.2. *Let $\mathbf{h}$ be a vector in $\mathbb{R}^d$ that is orthogonal to the ODT decision vectors $\mathbf{u}_i$ for all $i \in \{0, 1, \ldots, n\}$. Let the DLGN parameter $a > 0$. Let $\mathbf{w}_\lambda \cdot \mathbf{h} = 0$ for all $\lambda \in \{1, \ldots, L\}$. Let $\mathbf{w}_\lambda \cdot \mathbf{u}_j = 0$ for all $\lambda \in \{1, \ldots, L\}$ and $j \in \{N/2, \ldots, N\}$. The change in gradient of $\mathcal{L}$ w.r.t. $\mathbf{w}_k$ along $\mathbf{h}$ as $\mathbf{w}_\ell$ (for some $\ell \neq k$) moves along $-\mathbf{h}$ satisfies the following relation.*

$$\lim_{\epsilon \to 0} \frac{1}{\epsilon} \Big[-\nabla_{\mathbf{w}_k} \mathcal{L}(W + \epsilon P_\ell(\mathbf{h})) + \nabla_{\mathbf{w}_k} \mathcal{L}(W) \Big] \cdot (-\mathbf{h}) > 0$$

In words, Theorem 4.2 says that if some neuron $\mathbf{w}_\ell$ moves along some $\mathbf{h}$, the tendency of other neurons $\mathbf{w}_k$ to move along $-\mathbf{h}$ increases. For example, if for some reason (due to say finite data imbalances or some initial perturbation), some neuron $\mathbf{w}_\ell$ is moved along a direction $\mathbf{h}$ that is 'useless' from a generalisation perspective (e.g. $\mathbf{h} \cdot \mathbf{u}_i = 0$ for all i) then other layers tend to move along $-\mathbf{h}$ and the component of the gating vectors along $\mathbf{h}$ and $-\mathbf{h}$ keep increasing. Informally, each neuron tries to correct for mistakes in some other layers' neuron, and becomes part of the mistake that other neurons try to correct. The problematic nature of this behaviour becomes apparent only in the multi-path DLGN with deeper ODT label functions. We will discuss this further in Section 5.1.

Theorems 4.1 and 4.2, are applicable at the zero initialization, i.e. $W = 0$. The orthogonality conditions on $\mathbf{h}, \mathbf{z}, \mathbf{w}_\lambda$ and $\mathbf{u}_j$ in Theorems 4.1 and 4.2 reduce the direct applicability of these Theorems. We conjecture that these Theorems can be extended to relax the assumptions from orthogonality to simply that the cosine similarity is $O(1/\sqrt{d})$ or that the gating vectors $\mathbf{w}_\lambda$ all have low norm. This analysis is beyond the scope of this paper. The insights from Theorems 4.1 and 4.2 are valid beyond their restrictive assumptions. A main reason for this is that the second order dynamics are merely responsible for pushing the parameter into an appropriate basin in the loss landscape, and hence do not need to hold strongly throughout training. For example, if $|\mathbf{w}_\kappa . \mathbf{u}_0|$ is already large for some κ, then there is no need for some other layer $\mathbf{w}_k$ to move along $\mathbf{u}_0$.

4.3. Trajectory Analysis for a Chosen Initialization

Let us consider the ODT problem with depth $D = 4$ in Figure 1, and a single path DLGN with $L = 4$ layers. We focus on the solutions to weight vectors $\mathbf{w}_\ell$ and freeze the scalar coefficient $a = 1$. In this section we analyse in detail how Theorem 4.1 and 4.2 determine the parameter trajectory under gradient flow from a given initialization.

Firstly note that $a > 0$ means the model can achieve non-trivial log loss (loss values less than $\log(2)$) only on positive labelled points, i.e. data reaching label nodes $15, 17, \ldots, 29$. All these label nodes lie on the left side of their parents. Thus, when a DLGN is initialized with $W = 0$ and $a > 0$, gradient descent pushes all $\mathbf{w}_k$ along $\sum_j -\mathbf{u}_j$, where j takes on leaf node indices $N/2$ to N. This happens till the parameter W reaches the saddle point W^* and stays there. The gradient has no component along any other direction due to symmetry.

But, the properties of the loss landscape $\mathcal{L}(W)$ (mainly relating to Theorem 4.1) cause interesting things to happen even under a slight perturbation in an appropriate direction. Let the parameter W be initialized to a perturbation of the fully symmetric zero initialization, such that $\mathbf{w}_1 = \mathbf{0}$, $\mathbf{w}_2 = -\epsilon\mathbf{v}$, $\mathbf{w}_3 = -\epsilon\mathbf{v} - \epsilon\mathbf{u}_7$, $\mathbf{w}_4 = -\epsilon\mathbf{v} - \epsilon\mathbf{u}_8$. for some small $\epsilon > 0$ and $\mathbf{v} = \mathbf{u}_7 + \mathbf{u}_8 + \mathbf{u}_9 + \mathbf{u}_{10}$. The parameter and gradient trajectories with this initialization is given in Figure 2. This infinitesimal push from $\mathbf{0}$ starts a feedback loop between the neurons in different layers. The perturbation directions above were carefully chosen based on Theorem 4.1. From the perspective of $\mathbf{w}_1$, all the other layers have moved along $-\mathbf{u}_7, -\mathbf{u}_8, -\mathbf{u}_9$ and $-\mathbf{u}_{10}$. Thus Theorem 4.1 says that $\mathbf{w}_1$ experiences an increased push towards $-\mathbf{u}_0$. Now consider the version of Theorem 4.1 with $\mathbf{h}$ and $\mathbf{z}$ swapped. This movement of $\mathbf{w}_1$ towards $-\mathbf{u}_0$ pushes all the other layers further along the leaf node ODT directions and hence completes a feedback loop. Meanwhile, the minor extra movement of $\mathbf{w}_3$ and $\mathbf{w}_4$ along $-\mathbf{u}_7$

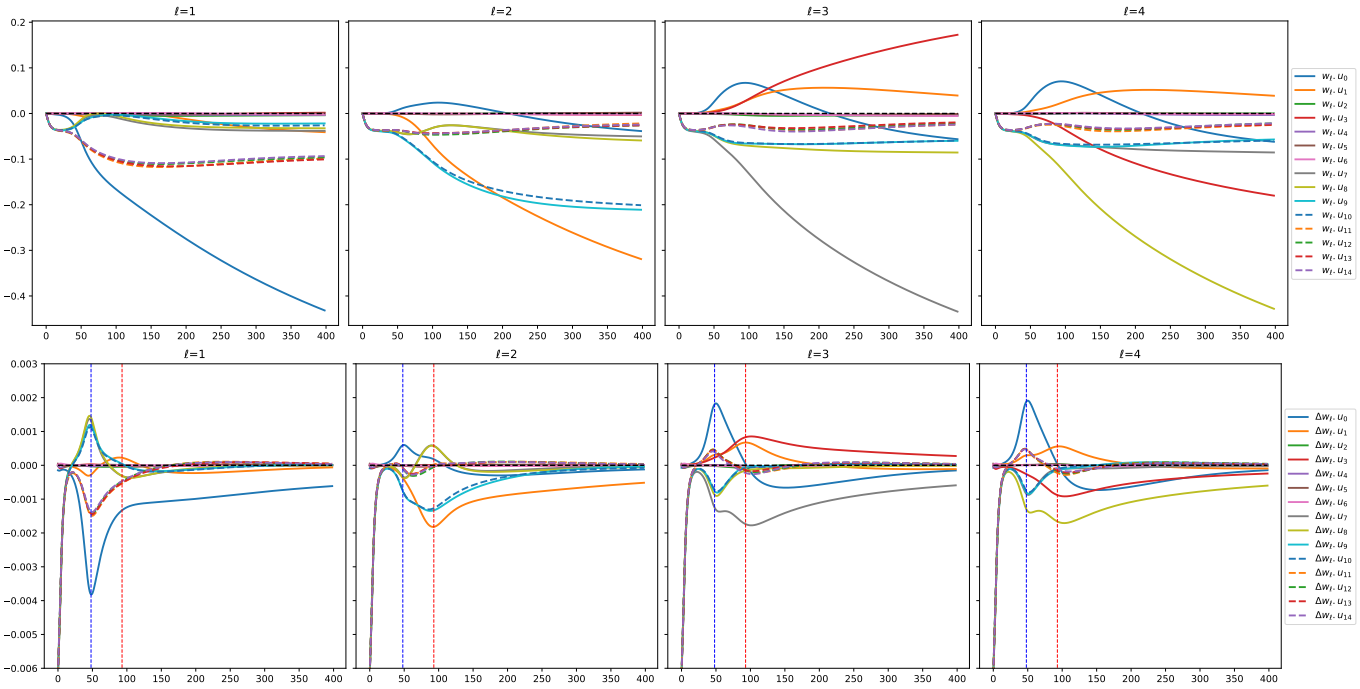

Figure 2. The parameter trajectory as a function of training time

and $-\mathbf{u}_8$ at initialization cause $\mathbf{w}_2$ to move along $-\mathbf{u}_1$ from Theorem 4.1. This tendency of $\mathbf{w}_2$ to move along $-\mathbf{u}_1$ is weak at initialization, and becomes stronger only after $\mathbf{w}_1$ has moved sufficiently along $-\mathbf{u}_0$. Also, the neurons $\mathbf{w}_k$ have a natural self-inhibitory tendency as indicated by Theorem 4.2, i.e. $\mathbf{w}_1$ moving along $-\mathbf{u}_0$ dissuades other neurons from doing the same. Even though all the neurons have an increased tendency to move along $-\mathbf{u}_0$ at initialization, this effect is strongest for $\mathbf{w}_1$.

Now we can read the parameter trajectories under gradient descent shown in Figure 2. The initialization scale ϵ is too small to be seen on the plot. The first 10 epochs correspond to the parameter movement from the initialization close to zero to the saddle point at W^*. The effect of second order dynamics from Theorem 4.1 begin to be visible only from this point. From around epoch 10 to 50 the gradient component of $\mathbf{w}_1$ along $\mathbf{u}_0$ keeps increasing. From around epoch 50 to 90, the gradient component of $\mathbf{w}_2$ along $\mathbf{u}_1$ keeps increasing. The neurons $\mathbf{w}_3$ and $\mathbf{w}_4$ which were closer to $-\mathbf{u}_7$ and $-\mathbf{u}_8$ at initialization grow along $-\mathbf{u}_7$ and $-\mathbf{u}_8$ respectively. The final solution gradient descent converges to is closely approximated (up to a scaling factor) by $\mathbf{w}_1 = -\mathbf{u}_0,\ \mathbf{w}_2 = -\mathbf{u}_1,\ \mathbf{w}_3 = -\mathbf{u}_7 + 0.3\mathbf{u}_3$ and $\mathbf{w}_4 = -\mathbf{u}_8 - 0.3\mathbf{u}_3$, as can be seen in Figure 5 in the Appendix. This corresponds to a superposition of two adjacent ODT paths to a positively labelled node – corresponding to $-\mathbf{u}_0, -\mathbf{u}_1, -\mathbf{u}_7, -\mathbf{u}_3$ and $-\mathbf{u}_0, -\mathbf{u}_1, +\mathbf{u}_3, -\mathbf{u}_8$. There are 4 such solutions, corresponding to the 2 leaf nodes being $7, 8$ or $9, 10$ or $11, 12$ or $13, 14$. Each of these 4 solutions can appear in $4! = 24$ configurations due to permutation invariance of the layers in the DLGN model. Thus making for a total of 96 solutions in the parameter space (or $2^{D-2}D!$ in general with $D = L$). One can easily design minor perturbations of $W = 0$ that reach any of these solutions under gradient descent.

We will use a 4-tuple to compactly represent the 96 solutions, based on the argmax of absolute value of the projections of the neurons along the ODT directions. i.e for a solution $\tilde{W} \in \mathbb{R}^{4\times d}$ it representation $\upsilon(\tilde{W}) \in \{0, 1, \ldots, 14\}^4$ is

$$\upsilon_\ell(\tilde{W}) = \text{argmax}_{i\in\{0,\ldots,14\}} |\mathbf{w}_\ell \cdot \mathbf{u}_i|.$$

For example, if $\tilde{W}$ is the convergent solution to the trajectory illustrated in Figure 2, then $\upsilon(\tilde{W}) = [0, 1, 7, 8]$.

4.4. Solution Prediction from Random Initialization

Constructing bespoke initializations close to zero that make parameter trajectory go to any of the solutions demonstrates a level of control and understanding over the gradient descent dynamics. However, in typical random initializations, the perturbations from $\mathbf{0}$ are not going to perfectly match one of these canonical perturbations. A typical scenario might have initialization supporting multiple solutions simultaneously. Assuming that gradient descent takes such a random initialization to one of the 96 solutions, which is it?

We do not have a full answer for this question. However, we can design heuristics that use Theorem 4.1 and 4.2 to 'measure' the support the initial parameter $W(0)$ gives to each of

the 96 solutions. In this section, we outline the construction of such a score, and perform experiments where the single path DLGN is trained from different random initializations. The details of this heuristic score construction are somewhat intricate and given in Appendix E.

We run gradient descent for the single path DLGN with 2000 different initializations, each with 5 different datasets sampled from the same distribution. This is done to ensure that the gradient descent convergent solution is purely a property of the initialization, and not any finite data effects. 368 of these initializations passed the test – i.e. the gradient descent convergent solution for these initializations did not change with data. The heuristic score was applied to the initialization, and used to predict the convergent solution. The parameters c_1, c_2, c_3 of the score function were searched from 0.0 to 0.9 in increments of 0.1 for a total of 1000 possible configurations. The best performing coefficients (which happened to be $c_1 = 0.1, c_2 = 0.7, c_3 = 0.4$) gave the correct solution for 179 out of these 368 initializations.

A baseline predictor that just predicts the solution nearest to initialization (corresponding to using the heuristic score function above with $c_1 = c_2 = c_3 = 0$) predicts accurately only for 46 out of the 368 initializations. In particular, the lens component of this heuristic score motivated from Theorems 4.1 and 4.2 is crucial to the score – zeroing its effects makes the heuristic score drop in accuracy from about 49% to 33%. This makes our notion of 'layers as lenses' a viable explanation of the learning dynamics of the DLGN.

5. The Multi-Path DLGN

In this section we consider the much more practical multi-path DLGN with $M > 1$. Setting $M = 1$ upper bounds the maximum representative power of the DLGN – it can at best capture ODT data entering one leaf node, which corresponds to low loss on $\frac{1}{2^D}$ fraction of the data, and close to random performance on the rest of the data. Larger width DLGN models do not suffer from this problem. For example, in this section we will consider a setting of 100 dimension, depth 5 ODT labelled data, and a 5-layer DLGN model having 16 nodes per layer, i.e $d = 100, D = 5, L = 5, M = 16$. It is possible to achieve losses and error rates on the entire data using such models. We note that no linear/kernel/tree based method perform satisfactorily (error rates remain $>$ 30%) for this data. Thus this data is an example that shows successful feature learning by deep models. Refer Appendix F for a detailed comparison of DLGN with other models on different ODT datasets.

5.1. Effect of Gating Regularization

Let the DLGN model be trained under a standard set-up where the log loss (over both the scaling parameters a and gating parameters $\mathbf{w}$) is minimized using a gradient descent method. This gives reasonable performance – the train loss goes down from $\log(2) = 0.69$ at initialization to around 0.05. The test error rate goes from 50% at initialization to about 10%. This would be the end point for a standard architecture (like the ReLU network), or some other dataset (like CIFAR10) as no further debugging/performance analysis is possible – only mindless hyperparameter tuning till we get better performance.

However, we are training a DLGN model on the ODT dataset, and hence we know exactly what the desirable learning behaviour is – as training progresses the gating vectors $\mathbf{w}_{m,\ell}$ should increase in norm along one of the ODT decision vectors $\mathbf{u}_j$. It is clear that any component of $\mathbf{w}_{m,\ell}$ along a direction orthogonal to all $\mathbf{u}_i$ can be of no help in learning the target function. Figure 3(b) gives a scatter plot showing how the $16 * 5 = 80$ neurons are at the end of training. The x-axis corresponds to the norm of the gating vector, and the y-axis corresponds to the max cosine similarity with any of the ODT vectors. A corresponding figure for the same before training begins is given in Figure 3(a). We can see that a significant fraction of the neurons have high norms, but do not correspond to any of the ODT vectors. This is clearly undesirable behaviour.

We conjecture that, a major component of this undesirable behaviour is from Theorem 4.2, where one neuron moving along a 'useless' direction makes other neurons move in 'useless' directions too. If this is indeed the reason for the undesirable behaviour, a potential fix might be to add an L2 penalty term for the gating norm to the objective, instead of pure cross entropy loss. Note that the L2 penalty term is not intended to be a regulariser for improving test performance, but rather it is designed to remove undesirable structures in the training loss optimisation landscape. This simple change to the objective causes a significant change in the optimization behavior. The resulting scatter plot showing gating vector norms and cosine similarity to the ODT decision vectors at the end of training with a L2 penalty is given in Figure 3(c). Almost all large norm gating vectors are aligned with one of the ODT decision vectors.

However, this L2 penalty restricts the gating vectors to even move along the ODT decision vectors, thus causing to converge to a worse training loss of 0.2 and a test error of 15%. We propose a natural modification to keep the benefits of regularization, while also letting the gating vectors grow in norm. We simply restart training with the L2 penalty convergent solution as the initialization, and remove the L2 penalty in this second phase of training. Additionally, during the initialisation we also forcibly set gating vectors

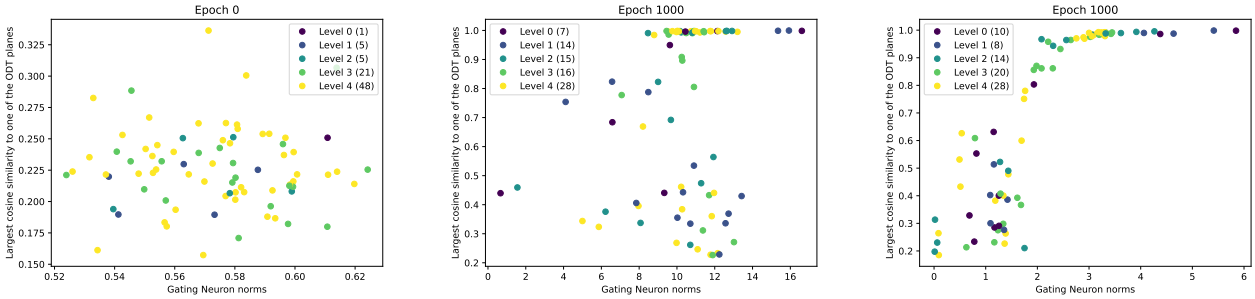

Figure 3. Gating norms vs cosine sim. to ODT vectors at (a) the beginning of training (b,c) end of training without and with L2 penalty.

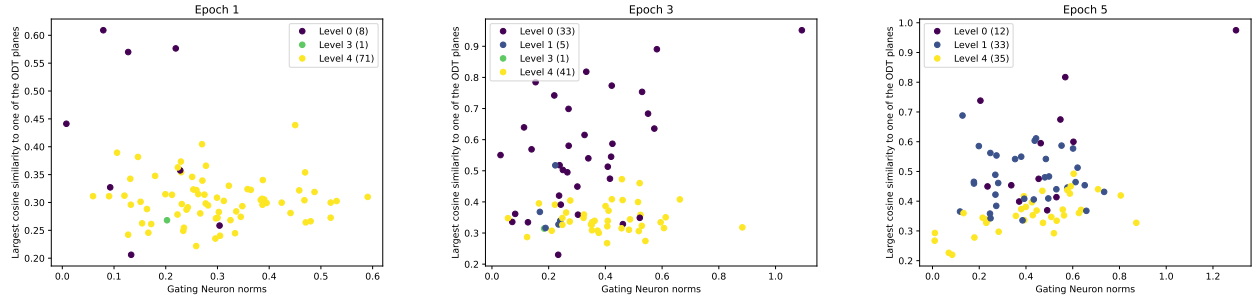

Figure 4. Gating norms and cosine similarity to ODT vectors during training (with the L2 penalty).

below a threshold norm to 0 and keep it fixed throughout the second phase of training. This modification is successful and reduces the test error further to around 5%. Note that this is a legitimate training strategy, as we never explicitly use the ODT vectors in the training procedure and only use it in the scatter plots to illustrate the problem and motivate a modification to standard gradient descent.

5.2. Hierarchical Feature Emergence

Looking into the learning dynamics of the multi-path DLGN with a L2 penalty on the gating vectors, we find interesting details that support our narrative of feature learning established in Section 4.3. Figure 2 suggests that the initial movement of the gating vectors is along the leaf node decision vectors until it reaches a saddle point, after which the ODT hierarchy kicks-in – i.e. the gating vectors move along the ODT root node vectors, then its children, and so on.

This hierarchical feature emergence can also be seen in the multi-path DLGN. Figure 4 shows the scatter plot of gating vector norms and cosine similarity to the ODT vectors at different points in training from a random initialization while training with a L2 penalty on the gating vectors. The individual gating neurons are colored based on the tree level of the nearest ODT decision vector. From a random initialisation (in Figure 3a), we can see a surge of neurons moving closer to the leaf nodes (Figure 4a) , then the root node is captured (Figure 4b), then its children (Figure 4c) and so on till the end when most of the ODT decision vectors are captured (Figure 3c).

This adds another candidate theory of feature emergence, usually based on spectral bias arguments motivated by the NTK. Of course, a general labeling function would not be a perfect COB-ODT and we will have to stretch our analogy to apply to this setting. One could view the hierarchy of the ODT mainly being present in the labeling function via a scope of discontinuity – the root node 0 is more prominent than its child node 1 because a significantly more data points (half exactly) would flip labels if $\mathbf{x} \cdot \mathbf{u}_0$ were to flip signs, than if $\mathbf{x} \cdot \mathbf{u}_1$ were to flip signs (a quarter exactly). This notion of scope of discontinuity is more applicable to general labeling functions, and could form a natural basis for a new theory on the dynamics of feature learning.

6. Conclusion

We propose a novel narrative of feature learning in deep networks that treats depth as an integral component of the model. We give theoretical support to it via second order behavior close to initialization, and provide empirical support via a detailed analysis of a single 'atom' of the deep learning architecture we study. Analysing the full architecture when an exponential number of these atoms are simultaneously learned is an exciting direction of future work.

7. Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Abbe, E., Boix-Adsera, E., Brennan, M. S., Bresler, G., and Nagaraj, D. The staircase property: How hierarchical structure can guide deep learning. In Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P., and Vaughan, J. W. (eds.), *Advances in Neural Information Processing Systems*, volume 34, pp. 26989–27002. Curran Associates, Inc., 2021.

Abbe, E., Adsera, E. B., and Misiakiewicz, T. The merged-staircase property: a necessary and nearly sufficient condition for sgd learning of sparse functions on two-layer neural networks. In Loh, P.-L. and Raginsky, M. (eds.), *Proceedings of Thirty Fifth Conference on Learning Theory*, volume 178 of *Proceedings of Machine Learning Research*, pp. 4782–4887. PMLR, 02–05 Jul 2022.

Abbe, E., Adserà, E. B., and Misiakiewicz, T. Sgd learning on neural networks: leap complexity and saddle-to-saddle dynamics. In Neu, G. and Rosasco, L. (eds.), *Proceedings of Thirty Sixth Conference on Learning Theory*, volume 195 of *Proceedings of Machine Learning Research*, pp. 2552–2623. PMLR, 12–15 Jul 2023.

Allen-Zhu, Z. and Li, Y. What can resnet learn efficiently, going beyond kernels? In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Allen-Zhu, Z. and Li, Y. Backward feature correction: How deep learning performs deep learning. *CoRR*, abs/2001.04413, 2020.

Arora, S., Cohen, N., and Hazan, E. On the optimization of deep networks: Implicit acceleration by overparameterization. In Dy, J. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 244–253. PMLR, 10–15 Jul 2018.

Arora, S., Du, S. S., Hu, W., Li, Z., Salakhutdinov, R., and Wang, R. On exact computation with an infinitely wide neural net. In *Neural Information Processing Systems (NeurIPS)*, 2019.

Atanasov, A., Bordelon, B., and Pehlevan., C. Neural networks as kernel learners. the silent alignment effect. In *In International Conference on Learning Representations*, 2022.

Ba, J., Erdogdu, M. A., Suzuki, T., Wang, Z., Wu, D., and Yang, G. High-dimensional asymptotics of feature learning: How one gradient step improves the representation. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 37932–37946. Curran Associates, Inc., 2022.

Banerjee, P., Ramaswamy, H. G., Yadav, M. L., and Lakshminarayanan, C. S. Deep networks learn features from local discontinuities in the label function. In *The Thirteenth International Conference on Learning Representations*, 2025.

Bordelon, B. and Pehlevan, C. Deep linear network training dynamics from random initialization: Data, width, depth, and hyperparameter transfer. In *Forty-second International Conference on Machine Learning*, 2025.

Chizat, L. and Bach, F. R. On the global convergence of gradient descent for over-parameterized models using optimal transport. In *Neural Information Processing Systems (NeurIPS*, 2018.

Chizat, L., Oyallon, E., and Bach, F. R. On lazy training in differentiable programming. In *Neural Information Processing Systems (NeurIPS)*, 2019.

Cui, H., Pesce, L., Dandi, Y., Krzakala, F., Lu, Y., Zdeborova, L., and Loureiro, B. Asymptotics of feature learning in two-layer networks after one gradient-step. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 9662–9695. PMLR, 21–27 Jul 2024.

Damian, A., Lee, J., and Soltanolkotabi, M. Neural networks can learn representations with gradient descent. In Loh, P.-L. and Raginsky, M. (eds.), *Proceedings of Thirty Fifth Conference on Learning Theory*, volume 178 of *Proceedings of Machine Learning Research*, pp. 5413–5452. PMLR, 02–05 Jul 2022.

Dandi, Y., Krzakala, F., Loureiro, B., Pesce, L., and Stephan, L. How two-layer neural networks learn, one (giant) step at a time. *Journal of Machine Learning Research*, 25 (349):1–65, 2024.

Daniely, A. and Malach, E. Learning parities with neural networks. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 20356–20365. Curran Associates, Inc., 2020.

Du, S., Lee, J., Li, H., Wang, L., and Zhai, X. Gradient descent finds global minima of deep neural networks.

In Chaudhuri, K. and Salakhutdinov, R. (eds.), *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 1675–1685. PMLR, 09–15 Jun 2019a.

Du, S. S., Zhai, X., Poczos, B., and Singh, A. Gradient descent provably optimizes over-parameterized neural networks. In *International Conference on Learning Representations*, 2019b.

Fort, S., Dziugaite, G. K., Paul, M., Kharaghani, S., Roy, D. M., and Ganguli, S. Deep learning versus kernel learning: an empirical study of loss landscape geometry and the time evolution of the neural tangent kernel. *Advances in Neural Information Processing Systems*, 33: 5850–5861, 2020.

Frei, S., Chatterji, N. S., and Bartlett, P. L. Random feature amplification: Feature learning and generalization in neural networks. *Journal of Machine Learning Research*, 24 (303):1–49, 2023.

Jacot, A., Gabriel, F., and Hongler, C. Neural tangent kernel: Convergence and generalization in neural networks. In Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

Kalimeris, D., Kaplun, G., Nakkiran, P., Edelman, B., Yang, T., Barak, B., and Zhang, H. Sgd on neural networks learns functions of increasing complexity. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Lee, J., Xiao, L., Schoenholz, S., Bahri, Y., Novak, R., Sohl-Dickstein, J., and Pennington, J. Wide neural networks of any depth evolve as linear models under gradient descent. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Li, Y., Ma, T., and Zhang, H. R. Learning over-parametrized two-layer neural networks beyond ntk. In Abernethy, J. and Agarwal, S. (eds.), *Proceedings of Thirty Third Conference on Learning Theory*, volume 125 of *Proceedings of Machine Learning Research*, pp. 2613–2682. PMLR, 09–12 Jul 2020.

Loo, N., Hasani, R., Amini, A., and Rus, D. Evolution of neural tangent kernels under benign and adversarial training. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems*, volume 35, pp. 11642–11657. Curran Associates, Inc., 2022.

Mei, S., Montanari, A., and Nguyen, P.-M. A mean field view of the landscape of two-layer neural networks. *Proceedings of the National Academy of Sciences*, 115(33): E7665–E7671, 2018. doi: 10.1073/pnas.1806579115.

Montufar, G. F., Pascanu, R., Cho, K., and Bengio, Y. On the number of linear regions of deep neural networks. In Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N., and Weinberger, K. (eds.), *Advances in Neural Information Processing Systems*, volume 27. Curran Associates, Inc., 2014.

Nichani, E., Damian, A., and Lee, J. D. Provable guarantees for nonlinear feature learning in three-layer neural networks. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 10828–10875. Curran Associates, Inc., 2023.

Soudry, D., Hoffer, E., Nacson, M. S., Gunasekar, S., and Srebro, N. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19 (70):1–57, 2018.

Telgarsky, M. Benefits of depth in neural networks. In Feldman, V., Rakhlin, A., and Shamir, O. (eds.), *29th Annual Conference on Learning Theory*, volume 49 of *Proceedings of Machine Learning Research*, pp. 1517–1539, Columbia University, New York, New York, USA, 23–26 Jun 2016. PMLR.

Wang, Z., Engel, A., Sarwate, A. D., Dumitriu, I., and Chiang, T. Spectral evolution and invariance in linear-width neural networks. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems*, volume 36, pp. 20695–20728. Curran Associates, Inc., 2023.

Wu, J., Braverman, V., and Lee, J. D. Implicit bias of gradient descent for logistic regression at the edge of stability. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

Yehudai, G. and Shamir, O. On the power and limitations of random features for understanding neural networks. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

Table 1. Table of notations

Variable and domain	Meaning
d	Data dimension
D	Depth of ODT tree
L	# of hidden layers in model
$\mathcal{L}$	Log loss for binary classification
M	# of neurons per hidden layer
$i, j, \iota, \zeta \in \{0, 1, \ldots, N\}$	ODT decision node indices, $N = 2^D - 2$
$\mu, \nu \in \{N+1, \ldots, 2N+2\}$	ODT label node indices
$p, q \in \{0, 1, 2, \ldots, D\}$	ODT depth indices
$k, l, \kappa, \lambda \in \{1, 2, \ldots, L\}$	DLGN Model layer indices
$m \in \{1, 2, \ldots, M\}$	Neuron index
$\pi \in \Pi = [M]^L$	DLGN path index
$\gamma \in \Gamma \subseteq \{0, \ldots, 2N+2\}^{D+1}$	ODT Path
$\gamma(\mathbf{x})$	ODT path traversed by point $\mathbf{x}$
$\sigma(a)$	$1/(1+\exp(-a))$
$\phi : \mathbb{R} \to [0, 1]$	Sigmoidal function.
$\tau \in \{+1, -1\}^D$	Sequence of left/right decisions
lev(i) $\in \{0, \ldots, D\}$	Distance of node i from 0 in the ODT
par$(i) \in \{0, 1, \ldots, N\}$	Parent of node i
anc$(i) \subseteq \{0, 1, \ldots, N\}$	Ancestors of node i
$\mathbf{v}, \mathbf{h}, \mathbf{z}$	Some vectors in $\mathbb{R}^d$
$\mathbf{u}_j \in \mathbb{R}^d$	ODT decision vector for node j
$\mathbf{w}_\ell \in \mathbb{R}^d$	DLGN parameter in layer ℓ

A. Notations

For any vectors $\mathbf{h}, \mathbf{z}$, We use both $\mathbf{h} \cdot \mathbf{z}$ and $\mathbf{h}^\top \mathbf{z}$ to both mean the dot product, and will use $\mathbf{h} \cdot \mathbf{z}$, whenever it is inconvenient to use $\mathbf{h}^\top \mathbf{z}$. Table 1 lists all the notations used in the paper.

B. Proof for Theorem 4.1

Proof. Without loss of generality, let $j = N/2$. Let $\gamma(\mathbf{x}) \in \{0, 1, \ldots, 2N+2\}^{D+1}$ be the path traversed in the ODT by data point $\mathbf{x}$. Its D-th element $\gamma_D(\mathbf{x})$ gives the leaf decision node, and its $D+1$-th element $\gamma_{D+1}(\mathbf{x})$ gives the label node. The label $y(\mathbf{x}) = +1$ if $\gamma_{D+1}(\mathbf{x})$ is odd, and -1 otherwise. Let $\epsilon > 0$ be an arbitrarily small number. All expressions involving ϵ below ignore quadratic and higher terms.

Let $\alpha(\mathbf{x}) = \frac{\phi(\mathbf{w}_\ell^\top \mathbf{x} + \epsilon \mathbf{h}^\top \mathbf{x})}{\phi(\mathbf{w}_\ell^\top \mathbf{x})} = 1 + \frac{\epsilon \mathbf{h}^\top \mathbf{x} \phi'(\mathbf{w}_\ell^\top \mathbf{x})}{\phi(\mathbf{w}_\ell^\top \mathbf{x})}$.

$$
\begin{aligned}
&-\nabla_{\mathbf{w}_k} \mathcal{L}(W + \epsilon P_\ell(\mathbf{h})) \cdot \mathbf{z} \\
&= \mathbf{E}_{\mathbf{x},y} \left[\sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x})) \Lambda_k(\mathbf{x}) \alpha(\mathbf{x}) \phi'(\mathbf{w}_k^\top \mathbf{x}) y a \mathbf{x} \cdot \mathbf{z} \right] \\
&= \frac{2}{2^D} \sum_{\iota=N/2}^{N} \mathbf{E}_{\mathbf{x},y} \left[\sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x})) \Lambda_k(\mathbf{x}) \alpha(\mathbf{x}) \phi'(\mathbf{w}_k^\top \mathbf{x}) y a \mathbf{x} \cdot \mathbf{z} \middle| \gamma_D(\mathbf{x}) = \iota \right] \qquad (4)
\end{aligned}
$$

We will analyze the sum of these conditional expectation terms separately for each ι. For each $\mathbf{x}'$, such that $\gamma_D(\mathbf{x}') = \iota$ and $\mathbf{h}^\top \mathbf{x}' > 0$ consider its reflection $\tilde{\mathbf{x}}$ along $\mathbf{h} = \mathbf{u}_j$ i.e. $\tilde{\mathbf{x}} = (I - 2\mathbf{u}_j \mathbf{u}_j^\top)\mathbf{x}'$. Let us consider a sum over just these two points

(instead of an expectation over all $\mathbf{x}, y$).

$$\sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x}))\Lambda_k(\mathbf{x})\alpha(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z}$$
$$= \sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \left(\sigma(-y\hat{y}(\mathbf{x})) + \epsilon\sigma'(-y\hat{y}(\mathbf{x}))\frac{\mathbf{h}^\top\mathbf{x}\phi'(\mathbf{w}_\ell^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})}\right)\Lambda_k(\mathbf{x})\left(1+\frac{\epsilon\mathbf{h}^\top\mathbf{x}\phi'(\mathbf{w}_\ell^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})}\right)\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z}$$
$$= \sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x}))\Lambda_k(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z} +$$
$$\frac{\epsilon\mathbf{h}^\top\mathbf{x}\phi'(\mathbf{w}_\ell^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})}\left(\sigma(-y\hat{y}(\mathbf{x})) + \sigma'(-y\hat{y}(\mathbf{x}))\,\Lambda_k(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z}\right) \tag{5}$$

Case 1: $\iota > N/2$. As $\iota \neq j$, we have that $\tilde{\mathbf{x}}$ and $\mathbf{x}'$ share the same ODT path, i.e. $\gamma(\tilde{\mathbf{x}}) = \gamma(\mathbf{x}')$, and hence the same label y. As the weights $\mathbf{w}_\lambda$ and the vector $\mathbf{z}$ are all orthogonal to $\mathbf{h} = \mathbf{u}_j$, all terms except $\mathbf{h}^\top\mathbf{x}$ are the same for both $\mathbf{x} = \mathbf{x}'$ and $\mathbf{x} = \tilde{\mathbf{x}}$. Further, $\mathbf{h}^\top\mathbf{x}' = -\mathbf{h}^\top\tilde{\mathbf{x}}$, thus making the second term in the above sum equal to zero. Thus Equation 5 becomes

$$\sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x}))\Lambda_k(\mathbf{x})\alpha(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z} = \sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x}))\Lambda_k(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z} \tag{6}$$

Case 2: $\iota = N/2$. For each $\mathbf{x}'$, such that $\gamma_D(\mathbf{x}') = \iota$ and $\mathbf{h}^\top\mathbf{x}' > 0$ consider its reflection $\tilde{\mathbf{x}}$ along $\mathbf{h} = \mathbf{u}_j$ i.e. $\tilde{\mathbf{x}} = (I - 2\mathbf{u}_j\mathbf{u}_j^\top)\mathbf{x}'$. As $\iota = j$, the point $\mathbf{x}'$ and $\tilde{\mathbf{x}}$ differ in their ODT paths – one of them terminates in label node $N+1$ and the other terminates in label node $N+2$, and have different labels y. Note that $\mathbf{z} = s\mathbf{u}_i$. Without loss of generality let j be in the left sub-tree of i, and $s = +1$. Thus $\mathbf{x}.\mathbf{z} < 0$ for both $\mathbf{x} = \mathbf{x}'$ and $\mathbf{x} = \tilde{\mathbf{x}}$. Further, we have $\mathbf{h} = \mathbf{u}_j$ where $j = N/2$ which is a leaf node, thus the label y for both $\mathbf{x} = \mathbf{x}'$ and $\mathbf{x} = \tilde{\mathbf{x}}$ is simply $\text{sign}(-\mathbf{h}\cdot\mathbf{x})$. Hence, $y\mathbf{h}^\top\mathbf{x}(\mathbf{x}.\mathbf{z}) > 0$. Equation 5 then becomes

$$\sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x}))\Lambda_k(\mathbf{x})\alpha(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z} > \sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x}))\Lambda_k(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot\mathbf{z} \tag{7}$$

Putting both the cases together, adding all the conditional expectation terms in Equation 4 using the inequalities in Equations 6 and 7, we get

$$-\nabla_{\mathbf{w}_k}\mathcal{L}(W + \epsilon P_\ell(\mathbf{h}))\cdot\mathbf{z} > -\nabla_{\mathbf{w}_k}\mathcal{L}(W)\cdot\mathbf{z}$$

The proof can be repeated almost verbatim for the case where $\mathbf{h}$ and $\mathbf{z}$ are swapped. □

C. Proof for Theorem 4.2

Proof. Let $\epsilon > 0$ be an arbitrarily small number. All expressions involving ϵ below ignore quadratic and higher terms. Let $\alpha(\mathbf{x}) = \frac{\phi(\mathbf{w}_\ell^\top\mathbf{x}+\epsilon\mathbf{h}^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})} = 1 + \frac{\epsilon\mathbf{h}^\top\mathbf{x}\phi'(\mathbf{w}_\ell^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})}$.

$$-\nabla_{\mathbf{w}_k}\mathcal{L}(W + \epsilon P_\ell(\mathbf{h}))\cdot(-\mathbf{h}) = \mathbf{E}_{\mathbf{x},y}\left[\sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x}))\Lambda_k(\mathbf{x})\alpha(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot(-\mathbf{h})\right]$$

For each $\mathbf{x}'$, such $y(\mathbf{x}') = 1$ consider its reflection $\tilde{\mathbf{x}}$ along $\mathbf{u}_\zeta$, where $\zeta = \gamma_D(\mathbf{x}')$ is the leaf node in the ODT corresponding to $\mathbf{x}'$. i.e. $\tilde{\mathbf{x}} = (I - 2\mathbf{u}_\zeta\mathbf{u}_\zeta^\top)\mathbf{x}'$. Let us consider a sum over just these two points (instead of an expectation over all $\mathbf{x}, y$). By construction, $y(\tilde{\mathbf{x}}) = -1$.

$$\sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x}))\Lambda_k(\mathbf{x})\alpha(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot(-\mathbf{h})$$
$$= \sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \left(\sigma(-y\hat{y}(\mathbf{x})) + \epsilon\sigma'(-y\hat{y}(\mathbf{x}))\frac{\mathbf{h}^\top\mathbf{x}\phi'(\mathbf{w}_\ell^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})}\right)\Lambda_k(\mathbf{x})\left(1+\frac{\epsilon\mathbf{h}^\top\mathbf{x}\phi'(\mathbf{w}_\ell^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})}\right)\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot(-\mathbf{h})$$
$$= \sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x}))\Lambda_k(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})ya\mathbf{x}\cdot(-\mathbf{h}) +$$
$$\frac{\epsilon\mathbf{h}^\top\mathbf{x}\phi'(\mathbf{w}_\ell^\top\mathbf{x})}{\phi(\mathbf{w}_\ell^\top\mathbf{x})}\left(\sigma(-y(\mathbf{x})\hat{y}(\mathbf{x})) + \sigma'(-y(\mathbf{x})\hat{y}(\mathbf{x}))\,\Lambda_k(\mathbf{x})\phi'(\mathbf{w}_k^\top\mathbf{x})y(\mathbf{x})a\mathbf{x}\cdot(-\mathbf{h})\right) \tag{8}$$

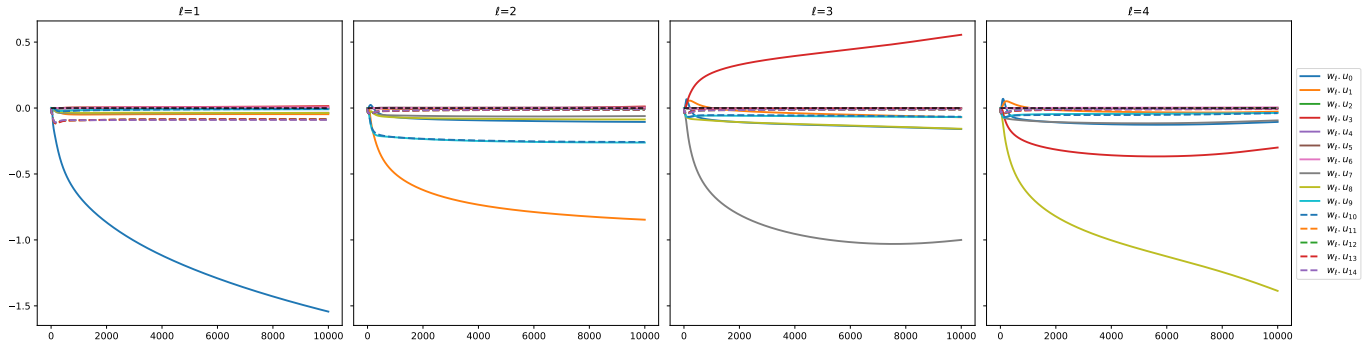

Figure 5. The parameter trajectory till 10000 epochs.

Note that $\mathbf{x}$ and $\mathbf{x}'$ differ only in their projections along $\mathbf{u}_\zeta$ which is orthogonal to all gating vectors $\mathbf{w}_\lambda$, and also $\mathbf{h}$. Even though $y(\mathbf{x}') = -y(\tilde{\mathbf{x}})$, and most of the other terms are equal for both $\mathbf{x}'$ and $\tilde{\mathbf{x}}$ the second term in Equation 8 does not become zero because of the $\sigma(-y(\mathbf{x})\hat{y}(\mathbf{x}))$ term. In more detail,

$$\begin{aligned}
&\sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \frac{\epsilon \mathbf{h}^\top \mathbf{x} \phi'(\mathbf{w}_\ell^\top \mathbf{x})}{\phi(\mathbf{w}_\ell^\top \mathbf{x})} \left(\sigma(-y(\mathbf{x})\hat{y}(\mathbf{x})) + \sigma'(-y(\mathbf{x})\hat{y}(\mathbf{x}))\, \Lambda_k(\mathbf{x}) \phi'(\mathbf{w}_k^\top \mathbf{x}) y(\mathbf{x}) a \mathbf{x} \cdot (-\mathbf{h})\right) \\
&= \frac{-\epsilon (\mathbf{h}^\top \mathbf{x}')^2 \phi'(\mathbf{w}_\ell^\top \mathbf{x}')}{\phi(\mathbf{w}_\ell^\top \mathbf{x}')} \Lambda_k(\mathbf{x}') \phi'(\mathbf{w}_k^\top \mathbf{x}') a \Big(y(\mathbf{x}') \sigma(-y(\mathbf{x}')\hat{y}(\mathbf{x}')) + y(\tilde{\mathbf{x}}) \sigma(-y(\tilde{\mathbf{x}})\hat{y}(\tilde{\mathbf{x}})) \Big) \\
&= \frac{-\epsilon (\mathbf{h}^\top \mathbf{x}')^2 \phi'(\mathbf{w}_\ell^\top \mathbf{x}')}{\phi(\mathbf{w}_\ell^\top \mathbf{x})} \Lambda_k(\mathbf{x}') \phi'(\mathbf{w}_k^\top \mathbf{x}') a \Big(\sigma(-\hat{y}(\mathbf{x}')) - \sigma(\hat{y}(\tilde{\mathbf{x}})) \Big) \qquad (9) \\
&> 0 \qquad (10)
\end{aligned}$$

The first equality above follows from observing σ' is an even function, and $y(\mathbf{x}') = -y(\tilde{\mathbf{x}})$, which zeros out the σ' term in the expansion of Equation 8, leaving only the σ term. The last inequality follows because $\hat{y}(\mathbf{x}') = \hat{y}(\tilde{\mathbf{x}}) > 0$ as $a > 0$, and thus $\sigma(-\hat{y}(\mathbf{x}')) - \sigma(\hat{y}(\tilde{\mathbf{x}})) < 0$.

Substituting Equation 9 into 8, we get

$$\begin{aligned}
&\sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x})\alpha(\mathbf{x})) \Lambda_k(\mathbf{x}) \alpha(\mathbf{x}) \phi'(\mathbf{w}_k^\top \mathbf{x}) y a \mathbf{x} \cdot (-\mathbf{h}) \\
&> \sum_{\mathbf{x}=\mathbf{x}',\tilde{\mathbf{x}}} \sigma(-y\hat{y}(\mathbf{x})) \Lambda_k(\mathbf{x}) \phi'(\mathbf{w}_k^\top \mathbf{x}) y a \mathbf{x} \cdot (-\mathbf{h})
\end{aligned}$$

and thus

$$-\nabla_{\mathbf{w}_k} \mathcal{L}(W + \epsilon P_\ell(\mathbf{h})) \cdot (-\mathbf{h}) > -\nabla_{\mathbf{w}_k} \mathcal{L}(W) \cdot (-\mathbf{h})$$

□

D. Single Path DLGN parameter trajectory

As an extension to Figure 2 in Section 4.3, Figure 5 shows the paramter trajectories when updated using gradient descent for 10000 epochs.

E. Heuristic Solution Predictor

The main component is a lens alignment score $A_{i,\ell}(W)$ that captures the tendency of $\mathbf{w}_\ell$ to move along $\mathbf{u}_i$ motivated from Theorem 4.1. It is based on the component of other layer neurons along $\mathbf{u}_j$ for some descendants or ancestors j of i

$$A_{i,\ell}(W) = \sum_{\gamma \in \Gamma_+ : i \in \gamma} \tau_{\psi(\ell)} \max_{\psi:[D]\to[D]} \sum_{k \neq \ell} \tau_{\psi(k)} \mathbf{w}_k \cdot \mathbf{u}_{\gamma_{\psi(k)}}$$

where Γ_+ corresponds to the set of ODT paths terminating in a positively labelled node. The τ referred to above is the $\{+1, -1\}$ valued decision vector associated with the path γ. A brief explanation of the above expression follows. Firstly, we consider all paths γ that contain the node i. For each such path γ, we consider all permutations ψ and add the contributions of other layer neurons along this permutation of the path γ. This corresponds to setting the measurement direction $\mathbf{z} = \mathbf{u}_i$ and considering all perturbation directions $\mathbf{h} = \mathbf{u}_j$ for ancestors or descendants j of i in Theorem 4.1.

For example, if $W = [\mathbf{0}, -\mathbf{u}_0, -\mathbf{u}_1, -\mathbf{u}_3]$, then $\mathbf{w}_1$ has a strong tendency to move along $-\mathbf{u}_7$ and make the DLGN represent the path $[0, 1, 3, 7, 15]$. This is reflected via a high negative score of -3 for $A_{7,1}(W)$. A breakdown of the calculation of $A_{7,1}(W)$ follows. $\gamma = [0, 1, 3, 7, 15]$ is the only path in Γ_+ that contains the node 7. Note that $\tau(\gamma) = [-1, -1, -1, -1]$. Out of all the 4! permutations of $\{1, 2, 3, 4\}$ the permutation $[4, 1, 2, 3]$ which calculates the alignment of W with $[-\mathbf{u}_7, -\mathbf{u}_0, -\mathbf{u}_1, -\mathbf{u}_3]$ scores the highest value of 3, and as $\tau_1 = -1$ the value of $A_{7,1}(W)$ becomes -3.

The lens alignment score is motivated by Theorem 4.1 and hence captures an element of the second order dynamics. We construct a node score that also takes into account zeroth and first order terms and also the self-mitigation term motivated from Theorem 4.2.

$$s_{i,\ell}(W) = \mathbf{w}_\ell \cdot \mathbf{u}_i - c_1 \mathbf{1}(i \geq N/2) + c_2 \sum_{k \neq \ell} (-\mathbf{w}_k \cdot \mathbf{u}_i) + c_3 A_{i,\ell}(W)$$

The first term above captures the current projection of $\mathbf{w}_\ell$ along $\mathbf{u}_i$. The second term captures the gradient close to initialization where significant gradient component is present only along leaf nodes $\mathbf{u}_{N/2}, \ldots, \mathbf{u}_N$ in the negative direction (recall $a > 0$). The third term captures the tendency of each layer $\mathbf{w}_\ell$ to move in a direction opposite to other layers $\mathbf{w}_k$ motivated from Theorem 4.2. The last term is $A_{i,\ell}(W)$. The coefficients c_1, c_2, c_3 are positive constants weighting the 4 different components. We now describe this term in greater detail below.

Finally, we convert the layer and tree node scores $s_{i,\ell}$ into solution scores using a simple additive rule. Let $\tilde{W}$ be one of the 96 solutions, and $v = v(\tilde{W})$ be the compact 4-tuple representation of $\tilde{W}$. The score given for solution $\tilde{W}$ at an initialization W is given by

$$S(W; \tilde{W}) = \sum_{\ell=1}^{4} s_{v_i,\ell}(W)$$

F. Performance comparison of different models on $D = 5$, $d = 100$ ODT dataset

We compare the performance of DLGN, ReLU and SVM on the depth D = 5, dimension d=100 ODT dataset containing 80,000 training points. Table 2 reports the best Test Accuracy (%) of each model.

Table 2. Comparison of Test Accuracy (%) on different datasets.

Dataset	**DLGN**	**ReLU**	**SVM**
d=100, D=5	94.92	95.74	63.27

Table 3. ReLU Hyper-parameters for L5 Dataset

Hyper-parameter	**Values**
Number of Hidden Layers	2, 3, **4**, 5, 6
Number of Hidden Neurons	20, **40**, 60, 80
Weight Decay	0, 0.001, **0.005**, 0.01, 0.05
Batch Size	**800**, 1600, 3200
Learning Rate	0.001, **0.005**, 0.01, 0.05

Table 4. SVM Hyper-parameters for L5 dataset

Hyper-parameter	Values
C	0.1, **1**, 10, 100
degree	2, 3, 4
gamma	**auto**, scale
kernel	polynomial, **RBF**, sigmoid, linear

F.1. Hyper-parameters for Experiments

This section contains the hyperparameter search space for the performance comparison experiments. The best hyperparameters are highlighted in bold letters. Table 3 show the list of hyper-parameters for ReLU experiments. Table 4 shows the list of hyper-parameters for SVM experiments.